CobaltStrike Malleable PE

HaCky 2022-07-15 09:42:00

二进制安全 木马与病毒

0x00 Malleable PE

Malleable PE 直译就是可拓展PE，通常来说，很多同学在做免杀的时候，会针对Loader进行免杀，并不会考虑针对beacon进行免杀，这就导致了很多杀软/EDR的内存防护能针对默认设置的beacon进行查杀。C2Profile提供了很好地操作beacon的方法，C2Profile不仅仅可以自定义beacon的通信属性(例如uri，header等等)，还可以对beacon进行操作，从而实现免杀的目的。

0x01 stage指标

C2Profile文件的stage块控制beacon的相关属性。如下图是官方文档提供的默认stage块。这个默认配置中，大概配置了一下几个属性，`userwx`,`complie_time`,`image_size_x86`,`image_size_x64`,`obfuscate`等等。这几个属性可以决定beacon部分PE属性和部分操作。

```
stage {
    set userwx "false";
    set compile_time "14 Jul 2009 8:14:00";
    set image_size_x86 "512000";
    set image_size_x64 "512000";
    set obfuscate "true";

    transform-x86 {
        prepend "\x90\x90";
        strrep "ReflectiveLoader" "DoLegitStuff";
    }

    transform-x64 {
        # transform the x64 rDLL stage
    }

    stringw "I am not Beacon";
}
```

`transform-x86`或者`transform-x64`可以修改反射dll的数据。这两个块支持3个命令，`prepend`,`append`,`strrep`。`prepend`是在beacon之前添加字节数据，这样可以规避那些单纯检测内存开头部分数据的EDR。`append`是在beacon数据之后添加额外的数据，而`strrep`则是替换字符串操作，其实很多厂商会将beacon的导出函数`ReflectiveLoader`作为检测的一个点，具体可以看flu0rite的记一次cs bypass卡巴斯基内存查杀，这篇文章详细介绍了卡巴针对beacon的内存查杀的特征，其中就有`ReflectiveLoader`。

命令	示例	作用
append	append "\x90\x90";	在Beacon反射 DLL最尾部的位置添加字符串
prepend	prepend "\x90\x90";	在Beacon反射DLL最开始的位置添加字符串
strrep	strrep "ReflectiveLoader" "DoLegitStuff";	替换Beacon反射 DLL 中指定的字符串

除此以外呢，stage还可以将字符串添加到beacon的.rdata段。

命令	示例	作用
string	string "xxx"	添加一个以零结尾的字符串
stringw	stringw "xxx"	添加一个宽（UTF-16LE 编码的）字符串
data	data "xxx"	按原样添加你的字符串

下面是针对stage块的一些配置属性。

- checksum：Beacon的PE的检验和
- cleanup：释放反射dll的内存
- compile_time：编译时间

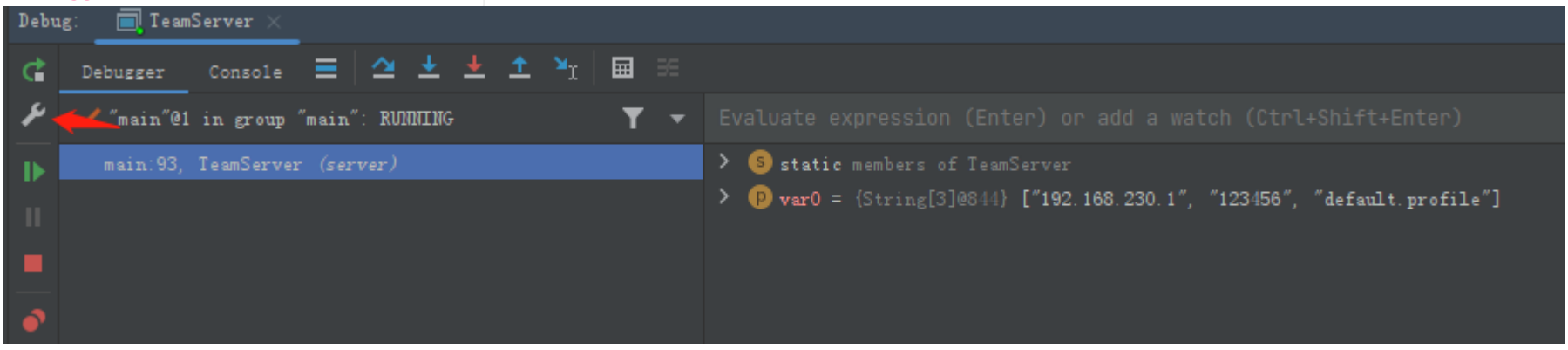
- entry_point：入口点
- image_size_x86(image_size_x64)：Beacon的PE的SizeOfImage(映像大小)
- module_x86(module_x64)：默认情况下，beacon是通过VirtualAlloc开辟内存的，如果设置module_x86属性，就可以Load module_x86指定的模块，然后将beacon复写到该模块的内存区域。
- name：决定beacon dll导出表的名字
- obfuscate：加密beacon Dll的导入表，复写未使用的header内容，以及让ReflectiveLoader将beacon拷贝到新的内存区域
- rich_header：rich_header是`PE`这个MagicNumber后面的pe数据
- sleep_mask：在sleep之前混淆beacon，避免内存查杀
- smartinject： Beacon 将关键函数指针（如 GetProcAddress 和 LoadLibrary）嵌入到同架构的post-ex DLL中。 这使 post-ex DLL可以在新进程中进行自我引导，而不会出现类似shellcode的行为，如使用PEB寻找kernel32.dll和其中的函数指针。
- stomppe：在加载完beacon之后，修改Characteristic值
- userwx：是否使用RWX权限，RWX容易引起怀疑
- allocator：控制Beacon反射加载器初始化时使用的内存分配函数（VirtualAlloc，HeapAlloc，MapViewOfFile）默认使用VirtualAlloc函数

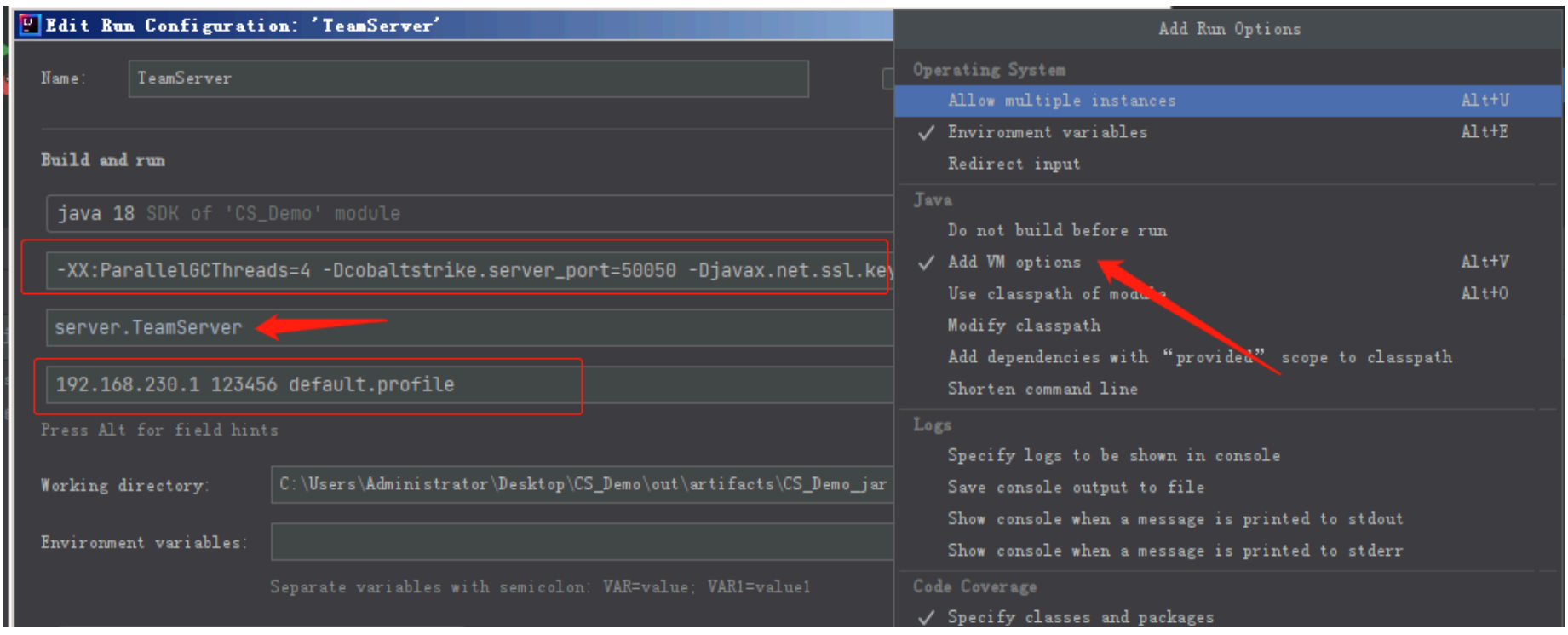
Option	Example	Description
checksum	0	The CheckSum value in Beacon's PE header
cleanup	false	Ask Beacon to attempt to free memory associated with the Reflective DLL package that initialized it.
compile_time	14 July 2009 8:14:00	The build time in Beacon's PE header
entry_point	92145	The EntryPoint value in Beacon's PE header
image_size_x64	512000	SizeOfImage value in x64 Beacon's PE header
image_size_x86	512000	SizeOfImage value in x86 Beacon's PE header
module_x64	xpsservices.dll	Same as module_x86; affects x64 loader
module_x86	xpsservices.dll	Ask the x86 ReflectiveLoader to load the specified library and overwrite its space instead of allocating memory with VirtualAlloc.
name	beacon.x64.dll	The Exported name of the Beacon DLL

obfuscate	false	Obfuscate the Reflective DLL's import table, overwrite unused header content, and ask ReflectiveLoader to copy Beacon to new memory without its DLL headers.
rich_header		Meta-information inserted by the compiler
sleep_mask	false	Obfuscate Beacon, in-memory, prior to sleeping
smartinject	false	Use embedded function pointer hints to bootstrap Beacon agent without walking kernel32 EAT
stomppe	true	Ask ReflectiveLoader to stomp MZ, PE, and e_lfanew values after it loads Beacon payload
userwx	false	Ask ReflectiveLoader to use or avoid RWX permissions for Beacon DLL in memory
allocator	VirtualAlloc	控制Beacon反射加载器初始化时使用的内存分配函数（VirtualAlloc，HeapAlloc，MapViewOfFile）默认使用VirtualAlloc函数

#0x02 C2Profile加载

TeamServer负责加载C2Profile文件，IDEA中找到server/TeamServer.java，在绿色三角启动调试，注意，在debug窗口中，点击扳手按钮，以设置调试选项。依次设置VM Optional，可以将TeamServer这个文件的设置直接复制进去，我这里设置的是`-XX:ParallelGCThreads=4 -Dcobaltstrike.server_port=50050 -Djavax.net.ssl.keyStore=./cobaltstrike.store -Djavax.net.ssl.keyStorePassword=123456 -XX:+AggressiveHeap -XX:+UseParallelGC`。然后添加主类，最后添加要调试的参数即可。





首先判断端口是否有效。然后加载C2Profile文件。

```
public static void main(String[] var0) {   var0: ["192.168.230.1", "123456", "default.profile"]
    int var1 = CommonUtils.toNumber(System.getProperty( key: "cobaltstrike.server_port",  def: "50050"), 50050);   var1: 50050
    if (!AssertUtils.TestPort(var1)) {   var1: 50050
        System.exit( status: 0);
    }
```

```
} else if (var0.length == 3 || var0.length == 4) {
    MudgeSanity.systemDetail("c2Profile", var0[2]);   var0: ["192.168.230.1", "123456", "default.profile"]
    Profile var6 = Loader.LoadProfile(var0[2]);
    if (var6 == null) {
        CommonUtils.print_error( var0: "exiting because of errors in " + var0[2] + ". Use ./c2lint to check the file");
        System.exit( status: 0) [Method will fail] ;
    }
```

将C2Profile文件按行读取，以'\n'分割。

```
    BufferedReader var2 = new BufferedReader(new InputStreamReader(var1));   var1: FileInputStream@1389    var2:
    StringBuffer var3 = new StringBuffer();   var3: "# default sleep time is 60s\nset sleeptime "60000";\n\n# jit

    String var4;   var4: null
    while((var4 = var2.readLine()) != null) {
        var3.append(var4);   var4: null
        var3.append('\n');   var3: "# default sleep time is 60s\nset sleeptime "60000";\n\n# jitter factor 0-99% [
```

然后对C2Profile进行解析，首先对Profile这个结构体进行初始化，这里面存储的是一些默认的C2Profile设置。然后调用Loader.parse进行解析读取到的C2Profile文件内容，

```
var3.addParameter(".amsi_disable", "false");   var3: Profile@1411
var3.addParameter(".sleeptime", "60000");
var3.addParameter(".jitter", "0");
var3.addParameter(".maxdns", "255");
var3.addParameter(".useragent", "<RAND>");
var3.addParameter(".spawnto", "rundll32.exe");
var3.addParameter(".spawnto_x86", "%windir%\\syswow64\\rundll32.exe");
var3.addParameter(".spawnto_x64", "%windir%\\sysnative\\rundll32.exe");
var3.addParameter(".pipename", "msagent_##");
var3.addParameter(".pipename_stager", "status_##");
var3.addParameter(".sample_name", CommonUtils.strrep((new File(var1)).getName(), ".profile", ""));
var3.addParameter(".dns_idle", "0.0.0.0");
var3.addParameter(".dns_sleep", "0");
var3.addParameter(".dns_stager_subhost", "");
var3.addParameter(".dns_stager_prepend", "");
var3.addParameter(".dns_max_txt", "252");
var3.addParameter(".dns_ttl", "1");
var3.addParameter(".tcp_port", "4444");
```

```
String var4;   var4: "# default sleep time is 60s\n"
String var5;   var5: "set"
String var6;   var6: "sleeptime"
String var7;   var7: ""60000""
String var9;
String var10;
if (var2 + 3 < var1.length) {
    var4 = var1[var2].toString();
    var5 = var1[var2 + 1].toString();
    var6 = var1[var2 + 2].toString();
    var7 = var1[var2 + 3].toString();   var1: Token[29]@1464    var2: 0
    if (Checkers.isSetStatement(var4, var5, var6, var7)) {   var4: "# default sleep time is 60s\n"
        if (!this.options.contains(var3 + "." + var5)) {
            this.parser.reportError( s: "invalid option for " + namespace(var3), var1[var2 + 1]);
```

这就是最后解析结果，最后在`TeamServer.go()`注册监听器(Listeners)，Beacons，Phisher等组件。最后建立监听，等待GUI进行链接。然后建立客户端的认证，然后拉起线程和客户端进行socket链接。

```
MudgeSanity.systemDetail("c2Profile", var0[2]);
Profile var6 = Loader.LoadProfile(var0[2]);   var0: ["192.168.230.1", "123456", "default.profile"]   var6: Profile@1411
if (var6 == null = false ) {   var6: Profile@1411
    CommonUtils.print_error( var0: "exiting because of errors in " + var0[2] + ". Use ./c2lint to chec
    System.exit( status: 0);
}

CommonUtils.print_good( var0: "I see you're into threat replication. " + var0[2] + " loaded.");
if (var0.length == 4 = false ) {
    long var4 = CommonUtils.parseDate(var0[3], "yyyy-MM-dd");
    if (var4 < System.currentTimeMillis()) {
```

```
var6 = {Profile@1411}
data = {HashMap@1450}  size = 94
".dns_idle" -> "0.0.0.0"
".code-signer.program_name" -> ""
".post-ex.spawnto_x64" -> "%windir%\sysnative\rundll32.exe"
"" -> {Program@1838}
".http-config.headers" -> ""
".http-post.server.output" -> {Program@1840}
".stage.name" -> "beacon.dll"
".http-get.server.output" -> {Program@1843}
```

```
CommonUtils.print_good( var0: "Team server is up on " + this.port);
CommonUtils.print_info( var0: "SHA256 hash of SSL cert is: " + var3.fingerprint());
this.resources.call( var1: "listeners.go");

while(true) {
    var3.acceptAndAuthenticate(this.pass, new PostAuthentication() {
        public void clientAuthenticated(Socket var1) {   var1: "SSLSocket[hostname=192.168.230.1, port=63249, Sessi
            try {
                var1.setSoTimeout(0);   var1: "SSLSocket[hostname=192.168.230.1, port=63249, Session(1656572520007|TL
                TeamSocket var2 = new TeamSocket(var1);
                (new Thread(new ManageUser(var2, TeamServer.this.resources, TeamServer.this.calls), name: "Manage: un
            } catch (Exception var3) {
                MudgeSanity.logException("Start client thread", var3, false);
            }

        }
    });
```

0x03 beacon差异性分析

0x3.1 module_x86

首先，采用默认的C2Profile生成beacon文件，至于生成beacon的流程可以参考这篇文章。然后测试一下`module_x86`属性。修改C2Profile文件的stage块。添加`set module_x86 "xpsservices.dll";`这个语句。module_x86这个语句的作用是加载module_x86指定的模块，然后将beacon复写到该模块的内存区域。

```
# this is the default profile. Make sure we look lik
stage {
    set stomppe "false";
    set name    "beacon.dll";
    set module_x86 "xpsservices.dll";
    string "%d.%s";
    string "post";
    string "%s%s";
    string "cdn.%x%x.%s";
    string "www6.%x%x.%s";
    string "%s.1%x.%x%x.%s";
```

在MalleablePE.java的`pre_process`会根据c2profile的set命令获取相对于的值，然后根据值对PE进行对于的操作。

```
2 usages
public byte[] pre_process(byte[] var1, String var2) {
    var1 = this.strings(var1);
    boolean var3 = this.profile.option( var1: ".stage.userwx");
    int var4 = this.profile.getInt( var1: ".stage.image_size_" + var2);
    String var5 = this.profile.getString( var1: ".stage.compile_time");
    boolean var6 = this.profile.option( var1: ".stage.obfuscate");
    String var7 = this.profile.getString( var1: ".stage.name");
    int var8 = this.profile.getInt( var1: ".stage.checksum");
    String var9 = this.profile.getString( var1: ".stage.module_" + var2);
    boolean var10 = this.profile.option( var1: ".stage.stomppe");
    String var11 = this.profile.getString( var1: ".stage.rich_header");
    int var12 = this.profile.getInt( var1: ".stage.entry_point");
    PEEditor var13 = new PEEditor(var1);
    var13.checkAssertions();
    if (!"<DEFAULT>".equals(var11)) {
        var13.insertRichHeader(CommonUtils.toBytes(var11));
    }
```

```
if (var10) {
    var13.stompPE();
}

if (var3) {
    var13.setRWXHint(var3);
}

if (!var5.equals("")) {
    var13.setCompileTime(var5);
}

if (var4 > 0) {
    var13.setImageSize((long)var4);
}

if (var8 > 0) {
    var13.setChecksum((long)var8);
}

if (!var7.equals("")) {
    var13.setExportName(var7);
}
```

在`setModuleStomp`这个方法中，首先将设置Characteristic为0x4000。然后在PE偏移为0x40的地方，添加大小为0x40的随机数据，接着在将C2Profile解析到的模块名(xpsservices.dll)写入PE偏移为0x40的地方，相当于重新覆盖直接写的随机数据的一部分。由此完成操作。

```
1 usage
public void setModuleStomp(String var1) {
    this.setCharacteristic(16384, true);
    this.setString(64, CommonUtils.randomData( var0: 64));
    this.setStringZ(64, var1);
}
```

对比一下生成的修改过module_x86属性的beacon.bin,和默认C2Profile生成的beacon.bin数据。有效的差异就是在PE+0x40出存在一个模块名。

```
040  0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68  .º..´.Í!¸.LÍ!Th
050  69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F  is program canno
060  74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20  t be run in DOS 
070  6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00  mode....$.......
080  A3 D2 AA 3F E7 B3 C4 6C E7 B3 C4 6C E7 B3 C4 6C  £Òª?ç³Älç³Älç³Äl
090  5A FC 52 6C E6 B3 C4 6C F9 E1 40 6C CF B3 C4 6C  ZüRlæ³Älùá@lÏ³Äl
0A0  F9 E1 51 6C F3 B3 C4 6C F9 E1 47 6C 65 B3 C4 6C  ùáQló³ÄlùáGle³Äl
0B0  C0 75 BF 6C EC B3 C4 6C E7 B3 C5 6C 2B B3 C4 6C  Àu¿lì³Älç³Ål+³Äl
0C0  F9 E1 4D 6C 2C B3 C4 6C F9 E1 56 6C E6 B3 C4 6C  ùáMl,³ÄlùáVlæ³Äl
0D0  F9 E1 55 6C E6 B3 C4 6C 52 69 63 68 E7 B3 C4 6C  ùáUlæ³ÄlRichç³Äl
0E0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

```
ers\Administrator\Documents\beacon_module_x86.bin
t(h)  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
000  4D 5A E8 00 00 00 00 5B 89 DF 52 45 55 89 E5 81  MZè....[‰ßREU‰å.
010  C3 B2 7C 00 00 FF D3 68 F0 B5 A2 56 68 04 00 00  Ã²|..ÿÓhðµ¢Vh...
020  00 57 FF D0 00 00 00 00 00 00 00 00 00 00 00 00  .WÿÐ............
030  00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00  ............ð...
040  78 70 73 73 65 72 76 69 63 65 73 2E 64 6C 6C 00  xpsservices.dll.
050  91 0F D6 70 0C 5E DB F7 2C 14 A2 E1 06 E8 82 4D  '.Öp.^Û÷,.¢á.è‚M
060  1F CA A3 D5 40 1C 91 45 CF A9 95 64 A4 37 81 13  .Ê£Õ@.'EÏ©•d¤7..
070  B4 8D AD 9B E6 01 9E 85 E2 9F 8E F4 FC 96 AD D3  ´..›æ.ž…âŸŽôü–.Ó
```

为了方便调试，生成一个beacon.exe程序，执行到beacon的时候，在PE偏移+0x40处下访问断点，以及针对LoadLibrary(Ex)W(A)下断点，程序中断在`003A82DF`处，每个断点都不一样。

```
3A82DB  83C0 40        add     eax, 40
3A82DE  50             push    eax
3A82DF  FF55 EC        call    dword ptr [ebp-14]          kernel32.LoadLibraryExA
3A82E2  8945 9C        mov     dword ptr [ebp-64], eax
3A82E5  C745 FC 010000 mov     dword ptr [ebp-4], 1
栈 ss:[00B2FF6C]=765A4813 (kernel32.LoadLibraryExA)

3A0000  4D 5A E8 00 00 00 00 5B 89 DF 52 45 55 89 E5 81  MZ?...[嗇REU文?
3A0010  C3 B2 7C 00 00 FF D3 68 F0 B5 A2 56 68 04 00 00  貌|..ÿ觝鸬  h¦..
3A0020  00 57 FF D0 00 00 00 00 00 00 00 00 00 00 00 00  .Wÿ?............
3A0030  00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00  .............?..
3A0040  78 70 73 73 65 72 76 69 63 65 73 2E 64 6C 6C 00  xpsservices.dll.
3A0050  F6 C4 BE 63 0C 99 AC B0 29 C9 65 9F 1A F8 19 06  瞿綾.槦?萄??■
3A0060  AC C0 0E 0B D8 6A E7 A3 F3 40 EC E0 8A 20 07 2F  绥■■豭纾驚爨?■/
3A0070  C0 55 70 A1 D2 80 87 50 53 0E 07 E3 5E 59 CB 12  繳p「■喇S■■鉤Y?

00B2FE80  003A0040  ASCII "xpsservices.dll"
00B2FE84  00000000
00B2FE88  00000001
00B2FE8C  003A0007  返回到 003A0007 来自 003A0007
00B2FE90  00000000
00B2FE94  00000000
00B2FE98  00000000
00B2FE9C  00000000
```

```
ProcAddress = 0;
if ( (*(v41 + 22) & 0x4000) != 0 && !(v42[0])(BaseAddr + 64) )
{
  hModule = (v42[3])(BaseAddr + 0x40, 0, 1);  // LoadLibraryEx
  for ( ProcIndex = 1; hModule != -1 && ProcIndex < 0x10 && !ProcAddress; ++ProcIndex )
    ProcAddress = (v42[1])(hModule, ProcIndex);// GetProcAddress(hModule_xpsservi,"1")
  if ( ProcAddress )                          // 将获取到的函数地址置空
  {
    ProcAddress -= ProcAddress % 0x1000;
    (v42[5])(ProcAddress, *(v41 + 80), v40, v43);// VirtualProtect
    memset(ProcAddress, 0, *(v41 + 80));
    (v42[5])(ProcAddress, *(v41 + 80), v40, v43);// VirtualProtect
  }
}
if ( !ProcAddress )
```

之后逻辑就很清楚了，首先加载指定的Module，然后通过函数需要，获取导出函数地址，分别将PE头和节区数据复制到目的地址。

```
003A87F1  8B45 DC      mov     eax, dword ptr [ebp-24]
003A87F4  8B4D C0      mov     ecx, dword ptr [ebp-40]
003A87F7  0348 28      add     ecx, dword ptr [eax+28]
003A87FA  894D C8      mov     dword ptr [ebp-38], ecx
003A87FD  8D7D E0      lea     edi, dword ptr [ebp-20]
003A8800  32C0         xor     al, al
003A8802  6A 18        push    18
003A8804  59           pop     ecx
003A8805  F3:AA        rep     stos byte ptr es:[edi]
003A8807  FF75 08      push    dword ptr [ebp+8]
003A880A  6A 01        push    1
003A880C  FF75 C0      push    dword ptr [ebp-40]
003A880F  FF55 C8      call    dword ptr [ebp-38]

ds:[003A0118]=00015888
ecx=629B1000 (xpsservi.629B1000), ASCII "MZ?

629B1000 4D 5A E8 00 00 00 00 5B 89 DF 52 45 55 89 E5 81 MZ?...[畜REU夊?
629B1010 C3 B2 7C 00 00 FF D3 68 F0 B5 A2 56 68 04 00 00 貌|..ÿ觝鸬  h¦..
629B1020 00 57 FF D0 00 00 00 00 00 00 00 00 00 00 00 00 .Wÿ?...........
629B1030 00 00 00 00 00 00 00 00 00 00 00 00 F0 00 00 00 ............?..
629B1040 78 70 73 73 65 72 76 69 63 65 73 2E 64 6C 6C 00 xpservices.dll.
629B1050 F6 C4 BE 63 0C 99 AC B0 29 C9 65 9F 1A F8 19 06 瞿綾.槤?萄??■
629B1060 AC C0 0E 0B D8 6A E7 A3 F3 40 EC E0 8A 20 07 2F 绥■■豭纾鸄爨?■/
629B1070 C0 55 70 A1 D2 80 87 50 53 0F 07 E3 5E 59 CB 12 繳p∫■嘲S■■鉤Y?
629B1080 A3 D2 AA 3F E7 B3 C4 6C E7 B3 C4 6C E7 B3 C4 6C Ｒ?绯腈绯腈绯腈
629B1090 5A FC 52 6C E6 B3 C4 6C F9 E1 40 6C CF B3 C4 6C Z麡l娉腈  @l铣腈
629B10A0 F9 E1 51 6C F3 B3 C4 6C F9 E1 47 6C 65 B3 C4 6C   Ql蟪腈  Gle衬l
629B10B0 C0 75 BF 6C EC B3 C4 6C E7 B3 C5 6C 2B B3 C4 6C 纔縧斐腈绯舕+衬l
629B10C0 F9 E1 4D 6C 2C B3 C4 6C F9 E1 56 6C E6 B3 C4 6C   Ml,衬l  Vl娉腈
629B10D0 F9 E1 55 6C E6 B3 C4 6C 52 69 63 68 E7 B3 C4 6C   Ul娉腈Rich绯腈
629B10E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
629B10F0 50 45 00 00 4C 01 04 00 5C 55 F2 5E 00 00 00 00 PE..L£¦.\U驄....
629B1100 00 00 00 00 E0 00 02 E1 0B 01 09 00 00 46 02 00 ....?¯?£...F¬.
629B1110 00 5C 01 00 00 00 00 00 88 58 01 00 00 10 00 00 .\£.....杢£..■..
629B1120 00 60 02 00 00 00 00 10 00 10 00 00 00 02 00 00 .`¬....■.■...¬..
629B1130 05 00 00 00 00 00 00 00 05 00 00 00 00 00 00 00 ¥.......¥.......
```

最后解析入口点，进行跳转即可

```
EC                  mov     [ebp+SizeOfHeaders], ecx
EF                  jmp     short loc_3A87FD
F1 ; ---------------------------------------------------------------------------
F1
F1 loc_3A87F1:                          ; CODE XREF: ReflectiveLoader(x)+B28↑j
F1                  mov     eax, [ebp+var_24]
F4                  mov     ecx, [ebp+ProcAddress]
F7                  add     ecx, [eax+_IMAGE_NT_HEADERS.OptionalHeader.AddressOfEntryPoint]
FA                  mov     [ebp+SizeOfHeaders], ecx
FD
FD loc_3A87FD:                          ; CODE XREF: ReflectiveLoader(x)+B36↑j
FD                  lea     edi, [ebp+var_20]
00                  xor     al, al
02                  push    18h
04                  pop     ecx
05                  rep stosb
07                  push    [ebp+arg_0]
0A                  push    1
0C                  push    [ebp+ProcAddress]
0F                  call    [ebp+SizeOfHeaders]
12                  mov     eax, [ebp+SizeOfHeaders]
15                  pop     edi
16                  pop     esi
17                  leave
```

0x3.2 stomppe

通过`set stomppe "true"`启用stomppe，看官方文档的介绍，stomppe的目的是轻微混淆内存中的Dll beacon，通过查看源码，通过了解`PEEditor.setCharacteristic`函数发现，该函数目的就是修改PE文件头中的Characteristic属性，这个属性和该文件的类型有关。用PEview查看默认生成的beacon文件，通过该属性值很显然的发现，这是一个32bit的dll可执行文件。感觉这个功能挺鸡肋的。

```
13 usages
protected void setCharacteristic(int var1, boolean var2) {   var1: 1    var2: true
    int var3 = this.getInfo().getLocation( var1: "Characteristics");   var3: 262
    if (var2) {   var2: true
        this.origch |= var1;
    } else {
        this.origch &= ~var1;   var1: 1
    }

    this.setShort(var3, (long)this.origch);   var3: 262    origch: 8451
}
```

00000100	00000000	Number of Symbols	
00000104	00E0	Size of Optional Header	
00000106	A102	Characteristics	
		0002	IMAGE_FILE_EXECUTABLE_IMAGE
		0100	IMAGE_FILE_32BIT_MACHINE
		2000	IMAGE_FILE_DLL
		8000	IMAGE_FILE_BYTES_REVERSED_HI

0x3.3 sleep_mask

sleep_mask的作用是在sleep之前加密beacon以规避基于内存特征的查杀。

中断在`BeaconPayload.setupGargle`方法处，首先，在setupGargle函数中，重新读取了resources/beacon.dll数据，然后将.text段的EndAddr保存在Settings的序号为41的地址处。

```
} else {
    PEParser var3 = PEParser.load(SleevedResource.readResource(var2));
    boolean var4 = this.c2profile.option( var1: ".stage.obfuscate");
    boolean var5 = this.c2profile.option( var1: ".stage.userwx");
    int var6 = var3.sectionEnd( var1: ".text");
    var1.addInt(41, var6);
```

然后计算rdata段和.text段之间的间距，如果间距小于256则不行。开始我很迷惑这步的操作，在后面的分析调试过程中，才发现，这个区域存储的是加密函数，而且当时我也好奇，如果要加密几个节区的话，代码段一旦被加密，事后怎么解密呢?其实CobaltStrike的设计者非常巧妙。

```
    boolean var5 = this.c2profile.option( var1: ".stage.userwx");
    int var6 = var3.sectionEnd( var1: ".text");
    var1.addInt(41, var6);
    int var7 = var3.sectionAddress( var1: ".rdata") - var6;  //
    if (var7 < 256) {
        CommonUtils.print_error( var0: ".stage.sleep_mask is true; nook space in " + var2 + " is " + var7 + " bytes. Beacon will crash.");
    }
```

最后获取了所有的Sections的起始地址和结束地址，并将其保存在序号为42的Settings处。

```
Packer var8 = new Packer();
var8.little();
if (!var4) {
   var8.addInt( var1: 0);
   var8.addInt( var1: 4096);
}

Iterator var9 = var3.SectionsTable().iterator();

while(true) {
   String var10;
   do {
      if (!var9.hasNext()) {
         var8.addInt( var1: 0);
         var8.addInt( var1: 0);
         var1.addData(42, var8.getBytes(), (int)var8.size());
         return;
      }

      var10 = (String)var9.next();
   } while(".text".equals(var10) && !var5);
```

生成并调试beacon.exe，通过之前的分析，sleep_mask作用就是在beacon 休眠之前加密beacon，所以只需要在beacon所存放的内存区域设置访问断点即可。具体可以这样操作，假设beacon分配在0x00300000处，首先，针对使程序运行到337D06处，这是程序暴力枚举到beacon区域起始地址，也就是0x00300000处。我们首先需要跳过这部分。然后在0x00300000处下访问断点，运行后断在0x003383D8处，

```
00337CF6   33C0          xor     eax, eax
00337CF8   40            inc     eax
00337CF9   74 44         je      short 00337D3F
00337CFB   8B45 94       mov     eax, dword ptr [ebp-6C]
00337CFE   0FB700        movzx   eax, word ptr [eax]
00337D01   3D 4D5A0000   cmp     eax, 5A4D
00337D06   75 2E         jnz     short 00337D36
00337D08   8B45 94       mov     eax, dword ptr [ebp-6C]
00337D0B   8B40 3C       mov     eax, dword ptr [eax+3C]
00337D0E   8945 98       mov     dword ptr [ebp-68], eax
00337D11   837D 98 40    cmp     dword ptr [ebp-68], 40
00337D15   72 1F         jb      short 00337D36
00337D17   817D 98 000400 cmp    dword ptr [ebp-68], 400
```

我们首先需要跳过这部分。然后在0x00300000处下访问断点，运行后断在0x003383D8处，显然可以看到依次将抹去了MZ的PE头和节区数据复制到新的区域。

```
}
else
{
  qmemcpy(k, v31, v36);                    // <----复制PE头
  if ( (*(v41 + 22) & 1) != 0 )
  {
    v31 = (*(k + offsetof(_IMAGE_DOS_HEADER, e_lfanew)) + v34);
    *v31 = 0;
    *k = 0;
    *(k + 60) = offsetof(_IMAGE_DOS_HEADER, e_magic);
  }
}
v36 = v41 + *(v41 + 20) + 24;
v37 = *(v41 + 6);
while ( v37-- )                            // <----复制节区数据
{
  v31 = (*(v36 + 12) + v34);
  k = *(v36 + 20) + v26;
```

```
堆栈 ss:[00A4FF48]=003301E8, (ASCII ".text")
eax=00024600

00550000 00 00 E8 00 00 00 00 5B 89 DF 52 45 55 89 E5 81 ..?...[壽REU夊?
00550010 C3 B2 7C 00 00 FF D3 68 F0 B5 A2 56 68 04 00 00 貌|..ÿ觝鸬  h¦..
00550020 00 57 FF D0 00 00 00 00 00 00 00 00 00 00 00 00 .Wÿ?...........
00550030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
00550040 0E 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 ■■?.???L?Th
00550050 69 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F is program canno
00550060 74 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 t be run in DOS
00550070 6D 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00 mode....$.......
00550080 A3 D2 AA 3F E7 B3 C4 6C E7 B3 C4 6C E7 B3 C4 6C Ｒ?绯腑绯腑绯腑
00550090 5A FC 52 6C E6 B3 C4 6C F9 E1 40 6C CF B3 C4 6C Z麡l娉腑  @l铣腑
005500A0 F9 E1 51 6C F3 B3 C4 6C F9 E1 47 6C 65 B3 C4 6C   Ql蟪腑  Gle衬l
005500B0 C0 75 BF 6C EC B3 C4 6C E7 B3 C5 6C 2B B3 C4 6C 纔縧斐腑绯舕+衬l
005500C0 F9 E1 4D 6C 2C B3 C4 6C F9 E1 56 6C E6 B3 C4 6C   Ml,衬l  Vl娉腑
005500D0 F9 E1 55 6C E6 B3 C4 6C 52 69 63 68 E7 B3 C4 6C   Ul娉腑Rich绯腑
005500E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
005500F0 00 00 00 00 4C 01 04 00 5C 55 F2 5E 00 00 00 00 ....L╞¦.\U驄....
00550100 00 00 00 00 E0 00 03 A1 0B 01 09 00 00 46 02 00 ....? ?╞...F╕.
00550110 00 5C 01 00 00 00 00 00 88 58 01 00 00 10 00 00 .\╞.....坌╞..■..
00550120 00 60 02 00 00 00 00 10 00 10 00 00 00 02 00 00 .`╕....■.■...╕..
00550130 05 00 00 00 00 00 00 00 05 00 00 00 00 00 00 00 ♪.......♪.......
```

然后在新的beacon的地址，也就是0x00550000处下访问断点。然后运行，此时中断在0x00575533。因为经过多次调试，图片的断点已经不相符了。显然，beacon被加密之后，进行休眠。

```
 E6        jnb     short 00715507
06         mov     eax, dword ptr [esi]
3C08       lea     edi, dword ptr [eax+ecx]
D2         xor     edx, edx
 07        push    7
C1         mov     eax, ecx
           pop     ebx
F3         div     ebx
4432 08    mov     al, byte ptr [edx+esi+8]
07         xor     byte ptr [edi], al
           inc     ecx
4D 08      cmp     ecx, dword ptr [ebp+8]
 E6        jb      short 00715521
45 FC      mov     eax, dword ptr [ebp-4]
 C7        jmp     short 00715507
75 10      push    dword ptr [ebp+10]
55 0C      call    dword ptr [ebp+C]          kernel32.Sleep
```

```
006F0000 05 F1 E7 56 AB DD DA 5E 78 D0 04 EE 88 53 E0 70 ♪╪ηV沃趋x?顒S郧
006F0010 CC E4 D7 DD DA FA 22 67 A6 1E 7F 8C 6D F5 0F 56 啼纵邡"g?■宫?V
006F0020 AB 8A 25 D5 F1 0F 56 AB DD DA 05 F1 0F 56 AB DD 珈%振■V沃??V沃
006F0030 DA 05 F1 0F 56 AB DD DA 05 F1 0F 56 AB DD DA 05 ??V沃??V沃?
006F0040 FF 10 EC A5 DD 6E 0C 3C 2E EE AA 91 17 24 A5 67 ÿ■欹輓.<.瞠?$
006F0050 3F D8 FD AA 77 9E 68 24 CA B0 FA 66 90 61 38 C4 ?佚猵谶$拾鶊恆8?
006F0060 A9 FA 67 94 2F 24 DE B3 FA 6C 9F 2F 12 E4 8E FA   g?$蕹鷏?■鋷?
006F0070 68 9E 6B 33 85 D0 D7 0F D5 0F 56 AB DD DA 05 F1 h瀫3哎??V沃??
006F0080 AC 84 01 E2 3D B6 35 63 B1 18 19 B6 E2 42 CB 3A 瑒┌??c?■垛B?
006F0090 F1 21 88 69 17 BC 92 C7 24 3B 45 9D C0 E5 6F B1 ?巫■黔?;E潟錹?
006F00A0 23 E4 A0 63 A5 18 19 B6 FC 10 48 3A CE 6E 1E 69 #鍎c?■饵■H:蝞■i
006F00B0 31 7A E9 C7 31 69 C1 9D E8 E5 6E B1 F1 B6 35 63 1z榍1i翅桢n熂?c
006F00C0 AF 4A 90 B6 29 42 CB 3A 52 3C 8C 69 17 BC 92 C7 疳惓)B?R<死■黔?
006F00D0 24 3B 50 9D E9 E5 6F B1 88 6C 92 67 B1 18 19 B6 $;P澀錹眻1抔?■?
006F00E0 05 F1 0F 56 AB DD DA 05 F1 0F 56 AB DD DA 05 F1 ♪?V沃??V沃??
006F00F0 0F 56 AB DD 96 04 F5 0F 0A FE 2F 84 05 F1 0F 56 ■V沃??.???V
006F0100 AB DD DA 05 11 0F 55 0A D6 DB 0C F1 0F 10 A9 DD 沃?■■U.舟.?■┴
006F0110 DA 59 F0 0F 56 AB DD DA 8D A9 0E 56 AB CD DA 05 搬?V沃攌?V庀?
006F0120 F1 6F 54 AB DD DA 05 E1 0F 46 AB DD DA 07 F1 0F 駉T沃??F沃??
006F0130 53 AB DD DA 05 F1 0F 56 AE DD DA 05 F1 0F 56 AB S沃??V镐??V?
```

当休眠结束之后，将加密的beacon还原。然后通过交叉引用，将主要执行这一操作的函数定位到DllMain函数的Sub_591391_OptionalFunc的Sub_594267_SleepMask函数。

```
59  75 04        jnz     short 0071555F
5B  85D2         test    edx, edx
5D  74 23        je      short 00715582
5F  3BCA         cmp     ecx, edx
61  73 E6        jnb     short 00715549
63  8B06         mov     eax, dword ptr [esi]
65  8D3C08       lea     edi, dword ptr [eax+ecx]
68  33D2         xor     edx, edx
6A  6A 07        push    7
6C  8BC1         mov     eax, ecx
6E  5B           pop     ebx
6F  F7F3         div     ebx
71  8A4432 08    mov     al, byte ptr [edx+esi+8]
75  3007         xor     byte ptr [edi], al
77  41           inc     ecx
78  3B4D 08      cmp     ecx, dword ptr [ebp+8]
7B  72 E6        jb      short 00715563
7D  8B45 0C      mov     eax, dword ptr [ebp+C]
80  EB C7        jmp     short 00715549
82  5F           pop     edi
83  5E           pop     esi
```

```
00590001 00 E8 00 00 00 00 5B 89 DF 52 45 55 89 E5 81 C3 .?...[蠆REU文你
00590011 B2 7C 00 00 FF D3 68 F0 B5 A2 56 68 04 00 00 00 瞸..ÿ觝鸬  h¦...
00590021 57 FF D0 00 00 00 00 00 00 00 00 00 00 00 00 00 Wÿ?............
00590031 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 0E ................
00590041 1F BA 0E 00 B4 09 CD 21 B8 01 4C CD 21 54 68 69 ▼?.???L?Thi
00590051 73 20 70 72 6F 67 72 61 6D 20 63 61 6E 6E 6F 74 s program cannot
00590061 20 62 65 20 72 75 6E 20 69 6E 20 44 4F 53 20 6D  be run in DOS m
00590071 6F 64 65 2E 0D 0D 0A 24 00 00 00 00 00 00 00 A3 ode....$.......?
00590081 D2 AA 3F E7 B3 C4 6C E7 B3 C4 6C E7 B3 C4 6C 5A 要?绯腑绯腑绯腑Z
00590091 FC 52 6C E6 B3 C4 6C F9 E1 40 6C CF B3 C4 6C F9 麡l娉腑  @l铣腑?
005900A1 E1 51 6C F3 B3 C4 6C F9 E1 47 6C 65 B3 C4 6C C0 酢l蟪腑  Gle衬l?
005900B1 75 BF 6C EC B3 C4 6C E7 B3 C5 6C 2B B3 C4 6C F9 u縧斐腑绯舕+衬l?
005900C1 E1 4D 6C 2C B3 C4 6C F9 E1 56 6C E6 B3 C4 6C F9 酠l,衬l  Vl娉腑?
005900D1 E1 55 6C E6 B3 C4 6C 52 69 63 68 E7 B3 C4 6C 00 酵l娉腑Rich绯腑.
005900E1 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ................
005900F1 00 00 00 4C 01 04 00 5C 55 F2 5E 00 00 00 00 00 ...L£¦.\U驄.....
00590101 00 00 00 E0 00 03 A1 0B 01 09 00 00 46 02 00 00 ...? ?£...F￢..
00590111 5C 01 00 00 00 00 00 88 58 01 00 00 10 00 00 00 \£.....杢£..■...
00590121 60 02 00 00 00 00 10 00 10 00 00 00 02 00 00 05 `￢....■.■...￢..¥
00590131 00 00 00 00 00 00 00 05 00 00 00 00 00 00 00 00 .......¥.......
```

```
BOOL __stdcall DllMain(HINSTANCE hinstDLL, DWORD fdwReason, LPVOID lpvReserved)
{
  struct _MEMORY_BASIC_INFORMATION Buffer; // [esp+0h] [ebp-1Ch] BYREF

  if ( fdwReason == 1 )
  {
    sub_599762(hinstDLL);
  }
  else if ( fdwReason == 4 )
  {
    if ( sub_599716() == 1 && hinstDLL && MEMORY[0x10026130](hinstDLL, &Buffer, 28) )
    {
      if ( Buffer.Type == 0x20000 )
      {
        MEMORY[0x100261F0](hinstDLL, 0, 0x8000);
      }
      else if ( Buffer.Type == 0x40000 )
      {
        MEMORY[0x100260D4](hinstDLL);
      }
    }
    Sub_591391_OptionalFunc();
  }
  return 1;
}
```

```
LABEL_22:
      if ( !MEMORY[0x1003219C] )
        return sub_599BB0();
    }
    v13 = MEMORY[0x1003219C] - v14;
LABEL_21:
    Sub_594267_SleepMask(v13);
    goto LABEL_22;
  }
  return sub_599BB0();
```

在Sub_8C9728_GetValueInC2Profile函数中，首先根据C2Profile的配置信息在偏移为0x29个元素的地方，取的值，如果取到的值为空的话，说明不需要加密，则直接Sleep，否则的话，进入Sub_8C4262_EncodeBeacon。

```
008C9707  C3              retn
008C9708  8B0D 88988F00   mov     ecx, dword ptr [8F9888]          c2Profile
008C970E  8B04D1          mov     eax, dword ptr [ecx+edx*8]       c2Profile+8*29h
008C9711  8B54D1 04       mov     edx, dword ptr [ecx+edx*8+4]
008C9715  C3              retn
008C9716  E8 EDFFFFFF     call    008C9708
008C971B  66:83F8 01      cmp     ax, 1
008C971F  75 04           jnz     short 008C9725
008C9721  66:8BC2         mov     ax, dx
008C9724  C3              retn
008C9725  33C0            xor     eax, eax
008C9727  C3              retn
008C9728  E8 DBFFFFFF     call    008C9708
008C972D  66:83F8 02      cmp     ax, 2                            取的type，判断是否为2
008C9731  75 03           jnz     short 008C9736
008C9733  8BC2            mov     eax, edx                         取Value
008C9735  C3              retn
```

```
Sub_594267_SleepMask proc near          ; CODE XREF: Sub_591391_OptionalFunc+232↑p

dwMilliseconds  = dword ptr  8

                push    ebp
                mov     ebp, esp
                push    29h ; ')'
                pop     edx
                call    Sub_8C9728_GetValueInC2Profile
                test    eax, eax        ; Value
                jz      short loc_8C429E ; 如果为空的话，则说明没有不需要进行加密，直接Sleep即可。
                cmp     dword ptr ds:10036824h, 0
```

Sub_8C4262_EncodeBeacon在经过jmp后，首先，获取10004262h和10004262h的差值，因为从进程中dump出来的beacon的符号信息不完整，我们重新生成beacon.bin文件，查看这部分信息。很显然，sub_10004262和sub_100041D5函数是连续的，其本质就是为了获取sub_100041D5函数地址。

```
            push    ebx
            push    esi
            push    edi
            push    29h ; ')'
            mov     esi, 10004262h
            mov     ebx, 100041D5h   ; 这两个函数在内存中是连续的，其实就是为了获取100041D5函数的大小
            pop     edx
            sub     esi, ebx         ; 相对偏移量，size
```

```
            push    edi
            push    29h ; ')'
            mov     esi, offset sub_10004262
            mov     ebx, offset sub_100041D5
            pop     edx              ; 29传入edx
            sub     esi, ebx         ; sub_10004262 - sub_100041D5
```

然后通过C2Profile获取.text段末尾的偏移地址，并将100041D5h的数据复制到.text段段尾。在重新生成的beacon.bin中查看100041D5h的内容。很显然其实就是一个加密函数。整个加密逻辑和上面其实是能对应上的。

```
        pop     edx
        sub     esi, ebx         ; 相对偏移量，size
        call    Sub_8C9728_GetIntValueInC2Profile ; 获取.text段末尾的偏移地址
        mov     ecx, eax
        call    Sub_8C9739_BaseAddr ; 获取beacon基地址
        mov     edi, ds:100260A4h
        add     ecx, eax         ; 相加，得到.text末尾地址
        lea     eax, [ebp+flOldProtect]
        push    eax              ; lpflOldProtect
        push    4                ; flNewProtect
        push    esi              ; dwSize
        push    ecx              ; lpAddress
        mov     ds:10035FCCh, ecx
        call    edi ; VirtualProtect
        push    esi              ; Size
        push    ebx              ; 这个函数是用来加密beacon的
        mov     ebx, ds:10035FCCh
        push    ebx              ; void *
        call    _memcpy_0
        add     esp, 0Ch
        lea     eax, [ebp+flOldProtect]
        push    eax              ; lpflOldProtect
        push    [ebp+flOldProtect] ; flNewProtect
        push    esi              ; dwSize
        push    ebx              ; lpAddress
        call    edi ; VirtualProtect
        push    10h              ; Size
        call     malloc
```

```
  unsigned int v6; // edx

  v3 = a1[1];
  while ( 1 )
  {
    v4 = *v3;
    v5 = v3[1];
    v3 += 2;
    if ( !v4 && !v5 )
      break;
    for ( ; v4 < v5; ++v4 )
      *(*a1 + v4) ^= *(a1 + v4 % 7 + 8);
  }
  a2(a3);
  result = a1[1];
  while ( 1 )
  {
    v7 = *result;
    v8 = result[1];
    result += 2;
    if ( !v7 && !v8 )
      break;
    for ( ; v7 < v8; ++v7 )
      *(*a1 + v7) ^= *(a1 + v7 % 7 + 8);
  }
  return result;
}
```

```
008E54FA   55            push   ebp
008E54FB   8BEC          mov    ebp, esp
008E54FD   51            push   ecx
008E54FE   53            push   ebx
008E54FF   56            push   esi
008E5500   8B75 08       mov    esi, dword ptr [ebp+8]
008E5503   8B46 04       mov    eax, dword ptr [esi+4]
008E5506   57            push   edi
008E5507   8B08          mov    ecx, dword ptr [eax]
008E5509   8B50 04       mov    edx, dword ptr [eax+4]
008E550C   83C0 08       add    eax, 8
008E550F   8955 08       mov    dword ptr [ebp+8], edx
008E5512   8945 FC       mov    dword ptr [ebp-4], eax
008E5515   85C9          test   ecx, ecx
008E5517   75 04         jnz    short 008E551D
008E5519   85D2          test   edx, edx
008E551B   74 23         je     short 008E5540
008E551D   3BCA          cmp    ecx, edx
008E551F   73 E6         jnb    short 008E5507
008E5521   8B06          mov    eax, dword ptr [esi]
```

接着逻辑就很简单了，申请一个长度为0x10的Buffer，然后偏移为0x00的地方存储BaseAddress，在+0x04的地方存储获取的那些节区的地址，最后在偏移为0x08的地方保存生成的Key。将加密函数EncodeFunc和保存上述信息的结构体放到两个全局变量中，就可以在函数外面使用了。

```
  Buffer = malloc(0x10u);
  dword_10035FD0 = Buffer;
  *Buffer = Sub_10009739_GetBaseAddr();
  *(Buffer + 1) = Sub_1000973F_GetDataValue(v5, 0x2A);
  Sub_1000A588_GetKey(Buffer + 8, 7u);
}
```

```
  int v1; // ecx

  if ( Sub_10009728_GetIntValue(v1, ')') && dword_10036824 <= 0 )
  {
    sub_10004262();
    (EncodeFunc)(Dword_10035FD0_Buffer, Sleep, dwMilliseconds);// Encode
  }
  else
  {
    Sleep(dwMilliseconds);
  }
}
```

将EncodeFunc下硬件执行断点，断下。整体逻辑大概就是这样的，首先去DataStruct的地址，这是一对地址(起始地址和结束地址)，分别取了地址之后，进行判空，以及起始地址要小于结束地址，然后去BaseAddress，因为DataStruct存储的是偏移地址，加上基地址才是各个区段的绝对地址。然后进行加密运算。当加密完成之后，执行Sleep操作，休眠完，在解密相关数据段即可。

```
    mov     esi, [ebp+arg_0]
    mov     eax, [esi+4]    ; 偏移+0x04 取DataStruect的地址
    push    edi

:                           ; CODE XREF: sub_100041D5+25↓j
                            ; sub_100041D5+44↓j
    mov     ecx, [eax]      ; 分别取起始地址StartAddr
    mov     edx, [eax+4]    ; 和结束地址EndAddr
    add     eax, 8          ; 指针指向下一组起始地址和结束地址(x86下两个地址长度为8)
    mov     [ebp+arg_0], edx
    mov     [ebp+var_4], eax
    test    ecx, ecx        ; StartAddr 判断是否为空
    jnz     short loc_100041F8
    test    edx, edx        ; EndAddr
    jz      short loc_1000421B

:                           ; CODE XREF: sub_100041D5+1D↑j
    cmp     ecx, edx
    jnb     short loc_100041E2 ; 是否起始地址小于结束地址

:                           ; CODE XREF: sub_100041D5+3F↓j
    mov     eax, [esi]      ; 取BaseAddress
    lea     edi, [eax+ecx]  ; 加上StartAddr，得到StartAddr的绝对地址
    xor     edx, edx
    push    7
    mov     eax, ecx
    pop     ebx
    div     ebx
    mov     al, [edx+esi+8]
    xor     [edi], al       ; 加密运算
    inc     ecx
    cmp     ecx, [ebp+arg_0] ; 判断起始地址是否等于结束地址EndAddr，两者都为offset
    jb      short loc_100041FC ; 取BaseAddress
    mov     eax, [ebp+var_4]
    jmp     short loc_100041E2 ; 分别取起始地址StartAddr
```

总结一下，CobaltStrike通过获取Section的起始地址和结束地址，并获取.text的结束地址，并将加密函数复制到.text的结束地址到rdata段之间空的地方，因为这个地方不会被加密。该加密函数内部会调用Sleep进行休眠，在休眠之前，加密数据，在休眠之后解密数据，以此往复，实现Sleep_Mask的功能。不得不说，设计的非常巧妙。

参考文献

- 记一次cs bypass卡巴斯基内存查杀
- https://wbglil.gitbook.io/cobalt-strike/cobalt-strikekuo-zhan/malleable-c2#malleable-pe-process-injection-and-post-exploitation beacon-hang-wei
- https://github.com/rsmudge/Malleable-C2-Profiles/blob/master/normal/reference.profile#L254-L255
- CobaltStrike4.0用户手册_中文翻译

评论

H

HaCky 2022-07-15 17:23:37

stomppe 这部分有点问题，和官方文档有出入，文档中介绍开启这项功能，则会混淆MZ头等PE头，但是在调试过程和生成的beacon数据中并未发现这个效果，不知道是怎么回事，有知道的大佬可以解答一下。

请先登录

你需要先登录后才能发表评论。

H